Faculty of Computing
Universiti Teknologi Malaysia
Data Structure & Algorithm

Tutorial
Create Class and Object using VS Code

Overview

- This tutorial will walk you through how to create class and object in C++ Program using Visual Studio Code

Prerequisites

1. Ensure Visual Studio Code is installed.
2. Install the C++ extension for Visual Studio Code.
3. Have a C++ compiler installed (e.g., MinGW for Windows, or use g++ for Linux/macOS).

Set Up Your Workspace

1. Open Visual Studio Code.
2. Create a new folder for your project and open it in VS Code.
3. Inside this folder, create a new file named main.cpp.

Part 1: Create a Simple Class

1. Open main.cpp.
2. Add the following code. Amend to write your details instead:

```cpp
#include <iostream>
using namespace std;

// Define a class called "AboutMe"
class AboutMe {
private:
    // Attributes
    string name;
    int age;
    string matricNumber;
    string address;

public:
    // Constructor
    AboutMe(string name, int age, string matricNumber, string address) {
        this->name = name;
        this->age = age;
        this->matricNumber = matricNumber;
        this->address = address;
    }

    // Method to display information
    void displayInfo() {
        cout << "Name: " << name << "\n" << "Age: " << age << "\n" <<"Matric Number:" << matricNumber << "\n" <<"Address:" <<address << endl;
    }
};

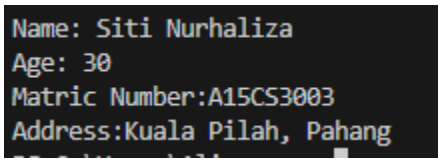
int main() {
    // Create an object of the AboutMe class
    AboutMe person1("Siti Nurhaliza", 30, "A15CS3003", "Kuala Pilah, Pahang");

    // Call the method to display the information
    person1.displayInfo();

    return 0;
}
```

Compile and Run

1. Press F5 to run the code.
2. The output will show the:



3. Upload all outputs in Elearning Forum. Reply to main thread created in the forum your name, matric no & the output image.

Part 2: Passing Class as Parameter

1. Open new .cpp file.
2. Add the following code:

```cpp
#include <iostream>
using namespace std;

// Define a simple class called "Car"
class Car {
private:
    string model;
    int year;

public:
    // Constructor
    Car(string model, int year) : model(model), year(year) {}

    // Method to display car information
    void displayInfo() const {
        cout << "Model: " << model << ", Year: " << year << endl;
    }

    // Method to set a new year
    void setYear(int newYear) {
        year = newYear;
    }
};

// Function that takes a Car object by value
void showCarByValue(Car car) {
    cout << "Pass by Value:" << endl;
    car.displayInfo(); // Displays the info of the copied object
    car.setYear(2022); // Modifies only the copied object
    car.displayInfo(); // Shows the change in the local copy
}

// Function that takes a Car object by reference
void showCarByReference(Car& car) {
    cout << "Pass by Reference:" << endl;
    car.displayInfo(); // Displays the info of the original object
    car.setYear(2023); // Modifies the original object
    car.displayInfo(); // Shows the change in the original object
}

// Function that takes a Car object by constant reference
void showCarByConstReference(const Car& car) {
    cout << "Pass by Const Reference:" << endl;
    car.displayInfo(); // Displays the info of the original object
    // car.setYear(2024); // Uncommenting this line will cause a compilation error (const prevents modification)
}
```

```
int main() {
    // Create a Car object
    Car car1("Toyota", 2020);

    // Pass by value
    showCarByValue(car1);
    cout << "Original after pass by value: ";
    car1.displayInfo(); // Original object remains unchanged

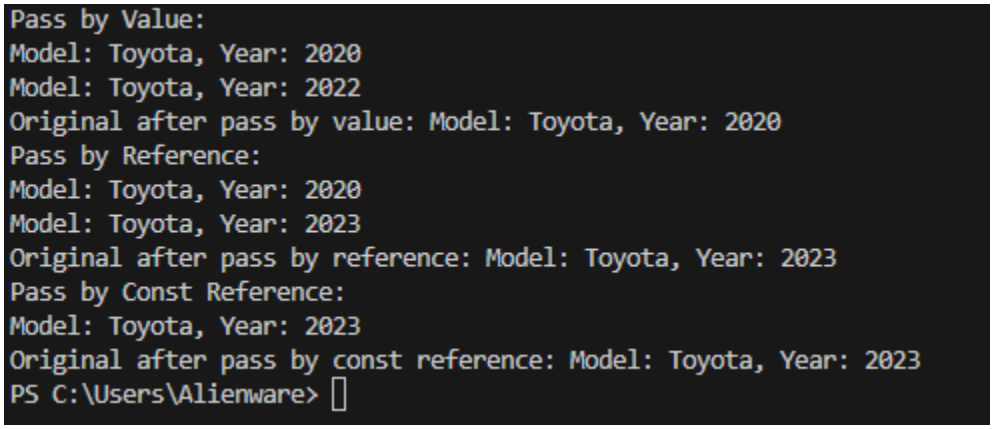
    // Pass by reference
    showCarByReference(car1);
    cout << "Original after pass by reference: ";
    car1.displayInfo(); // Original object is modified

    // Pass by const reference
    showCarByConstReference(car1);
    cout << "Original after pass by const reference: ";
    car1.displayInfo(); // Original object remains unchanged

    return 0;
}
```

Compile and Run

4. Press F5 to run the code.
5. The output will show the:



6. Upload all outputs in Elearning Forum. Reply to main thread created in the forum your name, matric no & the output image.

Explanation

1. Pass by Value (showCarByValue)
    - The function receives a copy of the Car object. Any changes made (like changing the year) affect only this copy, not the original object.
    - After calling showCarByValue, the original car1 remains unchanged.
2. Pass by Reference (showCarByReference)

- The function receives a reference to the original Car object, so any modifications (like changing the year) directly affect the original object.
- After calling showCarByReference, the changes are reflected in car1.

3. Pass by Const Reference (showCarByConstReference)
    - The function receives a constant reference to the Car object, meaning it can only read the object's data but cannot modify it.
    - This is useful when you want to prevent accidental modifications while still avoiding the overhead of copying the object.

Part 3: Passing Class as Parameter

1. Open new .cpp file.
2. Add the following code:

```cpp
#include <iostream>
using namespace std;

// Define a simple class called "Student"
class Student {
private:
    string name;
    int age;

public:
    // Default constructor
    Student() : name("Unknown"), age(0) {}

    // Parameterized constructor
    Student(string n, int a) : name(n), age(a) {}

    // Method to display student information
    void displayInfo() const {
        cout << "Name: " << name << ", Age: " << age << endl;
    }

    // Method to set student details
    void setDetails(string n, int a) {
        name = n;
        age = a;
    }
};
```

```
// Function to display information of an array of Student objects
void displayStudents(const Student* students, int size) {
    for (int i = 0; i < size; ++i) {
        cout << "Student " << i + 1 << ": ";
        students[i].displayInfo(); // Accessing members using array indexing
    }
}

int main() {
    // Create an array of 3 Student objects
    Student students[3] = {
        Student("Upin", 20),
        Student("Mei Mei", 22),
        Student("Jarjit", 19)
    };

    // Pointer to the array of students
    Student* ptr = students;

    // Iterate through the array using the pointer
    for (int i = 0; i < 3; ++i) {
        cout << "Student " << i + 1 << ": ";
        ptr->displayInfo(); // Using pointer to call the method
        ptr++; // Move pointer to the next object in the array
    }

    // Reset pointer back to the beginning of the array
    ptr = students;

    // Modify the details of the first student using the pointer
    ptr->setDetails("Mail", 21);

    // Display all students again to show the change
    cout << "\nAfter modifying the first student's details:\n";
    displayStudents(students, 3); // Using the new function to display students

    return 0;
}
```

Compile and Run

7. Press F5 to run the code.
8. The output will show the:

```
Student 1: Name: Upin, Age: 20
Student 2: Name: Mei Mei, Age: 22
Student 3: Name: Jarjit, Age: 19

After modifying the first student's details:
Student 1: Name: Mail, Age: 21
Student 2: Name: Mei Mei, Age: 22
Student 3: Name: Jarjit, Age: 19
PS C:\Users\Alienware>
```

9. Upload all outputs in Elearning Forum. Reply to main thread created in the forum your name, matric no & the output image.

Part 4: Exercise

1. Given the following program and the input, as well as the output of the program, fill in the space with the correct codes. Compile and run it.

Output:

Initial Inventory:
Device: Laptop, Price: 1500, Quantity: 10
Device: Phone, Price: 800, Quantity: 15
Device: Tablet, Price: 600, Quantity: 8

Total Inventory Value: 31800

After updating the first device:
Device: Gaming Laptop, Price: 2000, Quantity: 5
Device: Phone, Price: 800, Quantity: 15
Device: Tablet, Price: 600, Quantity: 8

```
1  #include <iostream>
2  #include <string>
3  using namespace std;
4
5  // Define the Device class
6  class Device {
7  private:
8      string name;
9      float price;
10     int quantity;
11
12 public:
13     // Constructor to initialize the attributes
14     Device(string n, float p, int q) {
15         _____________; // Fill in the blank to set name
16         _____________; // Fill in the blank to set price
17         _____________; // Fill in the blank to set
18                        quantity
19     }
```

```
20      // Method to calculate the total value of the device
21      float getValue() const {
22          return _____________; // Fill in the blank to
23  calculating the product of two private members that
24  determine the total value
25      }
26
27      // Method to display device information
28      void display() const {
29          cout << "Device: " << name
30               << ", Price: " << price
31               << ", Quantity: " << quantity << endl;
32      }
33
34      // Method to update the details of the device
35      void updateDetails(string n, float p, int q) {
36          name = n;
37          price = p;
38          quantity = q;
39      }
40  };
41
42  // Function to calculate the total value of the inventory
43  float calculateTotalValue(_______________) { // Fill in
44  the blank with parameters. Accepts two parameters, one is
45  an array of Device objects, and the other is the number of
46  elements
47      float total = 0;
48      for (int i = 0; i < size; ++i) {
49          total += _____________; // Fill in the blank to
50  add the value of each device
51      }
52      return total;
53  }
54
55  // Function to update a device using a pointer
56  void updateDevice(_______________, string n, float p, int
57  q) { // Fill in the blank with be a pointer to a 'Device'
58  object as parameter
59      _____________; // Fill in the blank to call the update
60  method that updates the device's details using the pointer
61  }
62
63
64
65
66
67
68
69
```

```
int main() {
    // Create an array of Device objects
    Device devices[3] = {
        Device("Laptop", 1500, 10),
        Device("Phone", 800, 15),
        Device("Tablet", 600, 8)
    };

    // Display all devices
    cout << "Initial Inventory:\n";
    for (int i = 0; i < 3; ++i) {
        devices[i]._____________; // Fill in the blank to call the display method
    }

    // Calculate and display the total inventory value
    float totalValue = calculateTotalValue(_______________); // Fill in the blank with arguments
    cout << "\nTotal Inventory Value: " << totalValue << endl;

    // Update the first device
    updateDevice(&devices[0], "Gaming Laptop", 2000, 5);

    // Display all devices again after the update
    cout << "\nAfter updating the first device:\n";
    for (int i = 0; i < 3; ++i) {
        devices[i]._____________; // Fill in the blank to call the display method
    }

    return 0;
}
```

10. Upload all outputs in Elearning Forum. Reply to main thread created in the forum your name, matric no & the output image.